

Software Architecture and Design Specification

Collaborative Study Material & Quiz Platform

Version 1.0

Status: Draft

04 October 2026

Group 5

Course: Software Engineering (UE24CS341A)

Faculty: Dr. Sudeepa Roy Dey, Associate Professor

Department of Computer Science and Engineering, PES University

Prepared by:

Bharat Shandilya (PES2UG24CS906)

Sandeep H (PES2UG24CS440)

Samyak Sanklecha (PES2UG24CS436)

Shreyas B Patil (PES2UG24CS488)

Submission Link

Submission Link: <https://forms.gle/hrG1rscDZfJbD7WV7>

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	Group 5	Initial architecture and design specification.

Approvals

Role	Name	Signature / Date
Team Member	Bharat Shandilya	
Team Member	Sandeep H	
Team Member	Samyak Sanklecha	
Team Member	Shreyas B Patil	
Faculty	Dr. Sudeepa Roy Dey	

1. Introduction

1.1 Purpose

This document describes the architecture and design of the Collaborative Study Material & Quiz Platform. It builds on the SRS and explains how the system is structured into components, how the main flows work, and the key design decisions we are following while building it.

1.2 Scope

The platform lets students share study material, maintain a topic-based question bank, create and attempt quizzes, track performance, and give feedback, with an admin role for basic moderation. The system runs on localhost for the demonstration. Public deployment, cloud hosting, payments, and third-party integrations are out of scope for this version.

1.3 Audience

Our team (as developers), and the course faculty and evaluator who review the design and the demo.

1.4 Definitions

MERN (MongoDB, Express.js, React, Node.js), SRS (Software Requirements Specification), STP (Software Test Plan), RTM (Requirements Traceability Matrix), REST, CRUD, RBAC (Role-Based Access Control), JWT (JSON Web Token).

2. Document Overview

2.1 How to use this document

Section 3 covers the architecture: the component diagram, the components, the chosen pattern, the technology stack, risks, requirement traceability, and security. Section 4 covers the design: sequence diagrams for the main flows, API design for two components, error handling, UX, and next steps.

2.2 Related Documents

- SRS - Collaborative Study Material & Quiz Platform, v1.0
- Project Proposal - Collaborative Study Material & Quiz Platform
- STP (Software Test Plan) - Collaborative Study Material & Quiz Platform, v1.0

3. Architecture

3.1 Goals & Constraints

Goals: keep the design simple and modular so the four of us can build our features in parallel, support role-based access (student and admin), and run reliably on localhost for the demo.

Constraints: the system is built on the MERN stack and runs locally only (no cloud or payment integrations); it targets roughly 20-30 concurrent users as stated in the SRS; and it must fit the semester timeline.

3.2 Stakeholders & Concerns

- Students - easy upload and browsing of material, attempting quizzes, and seeing their performance.
- Contributors - adding study material and questions to the shared bank.
- Admin - basic moderation of users, content, and topics.
- Faculty / Evaluator - correctness of the flows and a working demo on localhost.
- Developers (our team) - clear module boundaries so features can be built independently.

3.3 Component (UML) Diagram

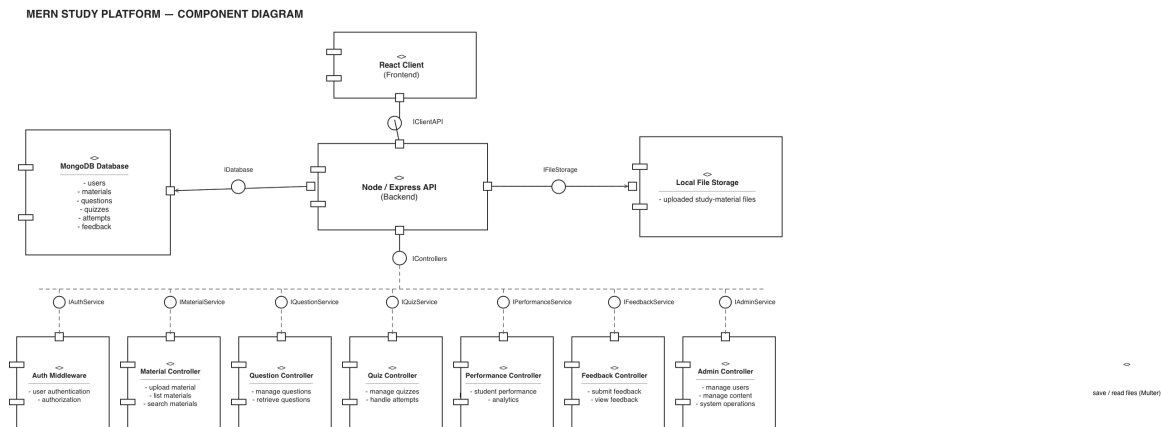


Figure 1. Component diagram of the MERN platform.

3.4 Component Descriptions

- React Client (Frontend) - the pages the user interacts with for each feature; talks to the backend over REST.
- Node / Express API (Backend) - exposes REST endpoints and holds the application logic.
- Auth Middleware - handles login and protects routes so only logged-in users can act; checks the role for admin actions.
- Material Controller - upload, list, and search study materials.
- Question Controller - add and retrieve questions in the bank.
- Quiz Controller - create quizzes from the bank and handle attempts and grading.
- Performance Controller - compute scores and topic-wise performance from saved attempts.
- Feedback Controller - submit and view ratings and feedback.
- Admin Controller - manage users, content, and topics.
- MongoDB Database - stores users, materials, questions, quizzes, attempts, and feedback.
- Local File Storage - stores uploaded study-material files on disk (via Multer).

3.5 Chosen Architecture Pattern and Rationale

We use a layered client-server architecture: a React presentation layer, an Express application layer exposing REST APIs, and a MongoDB data layer. We chose this because it gives a clear separation of concerns and lets each of us build a feature (model, controller, and page) on its own with minimal overlap. A microservices approach was considered but rejected as unnecessarily complex for a localhost academic project.

3.6 Technology Stack & Data Stores

- Frontend: React.
- Backend: Node.js with Express, using Mongoose to talk to MongoDB.
- Database: MongoDB - collections for users, materials, questions, quizzes, attempts, and feedback.
- File storage: uploaded study-material files are saved to a local folder using Multer.
- Auth: passwords are hashed with bcrypt, and a token/session-based auth middleware protects routes.

3.7 Risks & Mitigations

Risk	Mitigation
Delay in integrating modules before the demo	Integrate incrementally and test each module as it lands, rather than all at once.
Dependency chain (question bank needed before quizzes)	Lock the shared schemas early so dependent features can build against stub data.
Local environment differences across laptops	Keep a shared setup guide and use consistent Node and MongoDB versions.
Limited time in the semester	Prioritise the core features (REQ-1 to REQ-9) first.

3.8 Traceability to Requirements

Requirements	Component
REQ-1 to REQ-3 (study material)	Material Controller + material pages
REQ-4 to REQ-6 (question bank)	Question Controller + question pages
REQ-7 to REQ-9 (quizzes)	Quiz Controller + quiz pages
REQ-10 to REQ-12 (performance)	Performance Controller + dashboard
REQ-13 to REQ-14 (feedback)	Feedback Controller + rating widget
REQ-15 to REQ-17 (admin)	Admin Controller + Auth middleware (role)

3.9 Security Architecture

We apply the STRIDE threat model at a level that matches a localhost academic build:

Threat	How we handle it
Spoofing	Login with bcrypt-hashed passwords and auth middleware.

Tampering	Server-side input and file-type validation; writes go through authenticated routes.
Repudiation	Basic server-side request logging (no sensitive data in logs).
Information disclosure	Passwords are never stored in plain text; protected routes require login.
Denial of Service	Not addressed in this version (closed localhost demo environment).
Elevation of privilege	Role field plus admin-only route checks (RBAC).

4. Design

4.1 Design Overview

The design follows the layered architecture: each feature is a self-contained slice made of a Mongoose model, an Express controller with its routes, and a React page. All features share the auth middleware and the user role field, so access control is consistent across modules.

4.2 UML Sequence Diagrams

The two diagrams below cover the two central flows of the platform.

sequence diagram for quiz attempt and auto evaluation

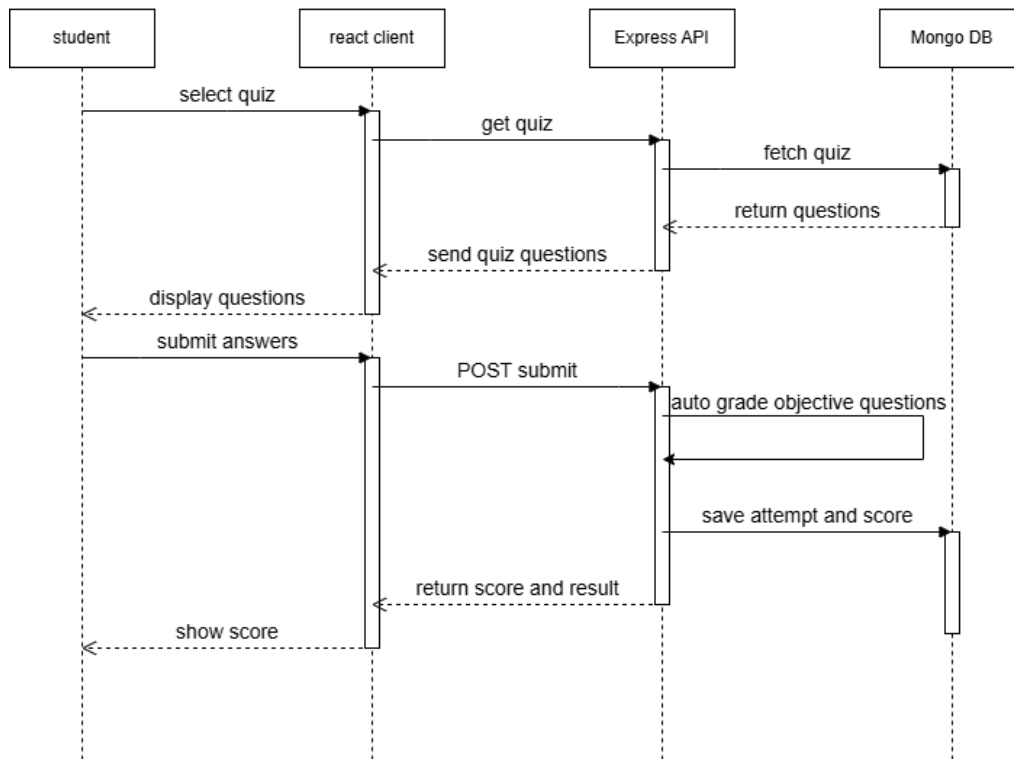


Figure 2. Quiz attempt and auto-evaluation.

The student selects a quiz, the client fetches it and its questions from the backend, the student submits answers, the backend auto-grades the objective questions, saves the attempt with its score, and returns the result.

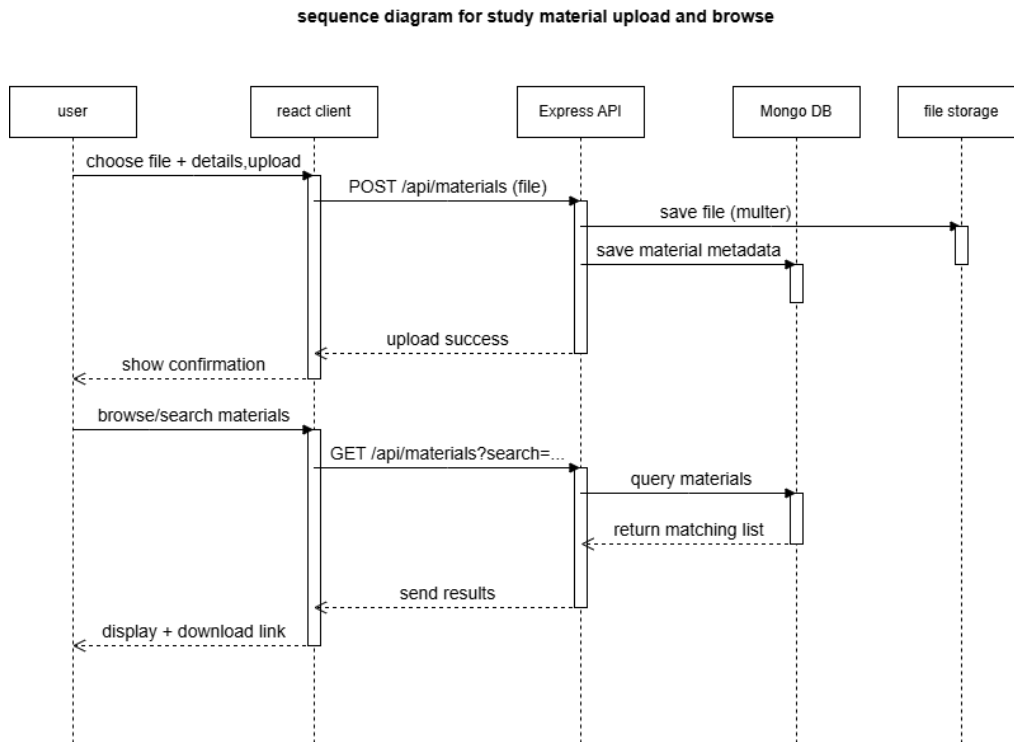


Figure 3. Study material upload and browse.

A user uploads a file with its details; the backend saves the file to local storage and the metadata to MongoDB. Later, a user browses or searches materials and the backend returns the matching list with a download link.

4.3 API Design

Interface definitions for two components.

Study Material API

Endpoint	Method	Description	Main responses
/api/materials	POST	Upload a material (file + title, subject, topic). Auth required.	201 created; 400 invalid/unsupported file type; 401 not logged in
/api/materials	GET	Browse / search / filter materials by subject or topic.	200 list of materials
/api/materials/:id	GET	Open or download a single material.	200 material; 404 not found

Quiz API

Endpoint	Method	Description	Main responses
/api/quizzes/:id	GET	Fetch a quiz and its questions. Auth required.	200 quiz; 404 not found
/api/quizzes/:id/submit	POST	Submit answers; the backend auto-grades and saves the attempt.	200 score and result; 400 invalid payload; 401 not logged in

4.4 Error Handling, Logging & Monitoring

- The API returns standard HTTP status codes with a short JSON error message (e.g. 400 bad input, 401 not logged in, 403 forbidden, 404 not found, 500 server error).
- File uploads with an unsupported type are rejected with a clear error shown on the form.
- The backend logs requests and errors on the server console, without writing sensitive data such as passwords.
- Monitoring is manual during the localhost demo; no external monitoring tool is used in this version.

4.5 UX Design

- A common navbar gives consistent navigation across pages.
- Forms show clear error states (for example on a wrong file type or a failed upload).
- The performance dashboard presents scores and topic-wise results in a simple, readable way.

4.6 Open Issues & Next Steps

- Build the foundation and auth first, then the core features, following the phased plan in the build plan.
- Integrate all modules and test the full journey on localhost before the review.
- Re-calibrate story-point estimates after the first sprint based on actual velocity.

5. Appendices

5.1 Glossary

MERN, REST, CRUD, RBAC, JWT, bcrypt, Multer, SRS, STP, RTM.

5.2 References

- SRS - Collaborative Study Material & Quiz Platform, v1.0
- Project Proposal - Collaborative Study Material & Quiz Platform

- STP - Collaborative Study Material & Quiz Platform, v1.0
- Course guidelines, Software Engineering (UE24CS341A)

5.3 Tools

draw.io (UML diagrams), Visual Studio Code, GitHub, Postman, MongoDB Compass.